Homomorphic Encryption

Security relevance

Homomorphic Encryption is an advanced cryptographic technique. Used for mathematic operations and processing data results. The twist is when processing data the entire data is encrypted during the entire process.

In the traditional usage of encryption, the data is protected when static via symmetric primitives such as AES. For in transit, it uses protocols like TLS. The problem is the application for data in use vulnerability where cyphertexts must be decrypted to plain text for calculation execution. Exposing sensitive data to unauthorized memory or insider threats or from cloud providers.

Architectural Workflow

- Data is encrypted locally via the user to protect from untrusted networks it encounters when in transit.
- The encrypted/scrambled data then sent to an untrusted environment for example a cloud server.
- The cloud server runs the calculations directly with the encrypted/cyphertext.
- Proceeding to output an encrypted result
- Once sent back to the user they use a private key to decrypt the outcome as if it were never encrypted at all. Thus, working completely in the background.

Ethical & Legality

Homomorphic Encryption enables for a stronger security for sensitive data. Ensuring for a zero-trust basis for protection. This kind of encryption opens the door to enable machine learning training framework and much more, specifically not violating GDPR which significantly changes the opportunity and approach for financial services or healthcare. Keeping a stronger and firmer way of preventing outside environment from viewing this kind of information.

Although this is all great when sending to a cloud or an unsecure environment. There are still some vulnerabilities such as inference/leakage attacks. Also, if there are only two employees within one department and they know how much they make, they could deduce what they make. Alongside this if a rogue user within the company decide to change data within the database this can easily mess up and cause all kinds of problems.

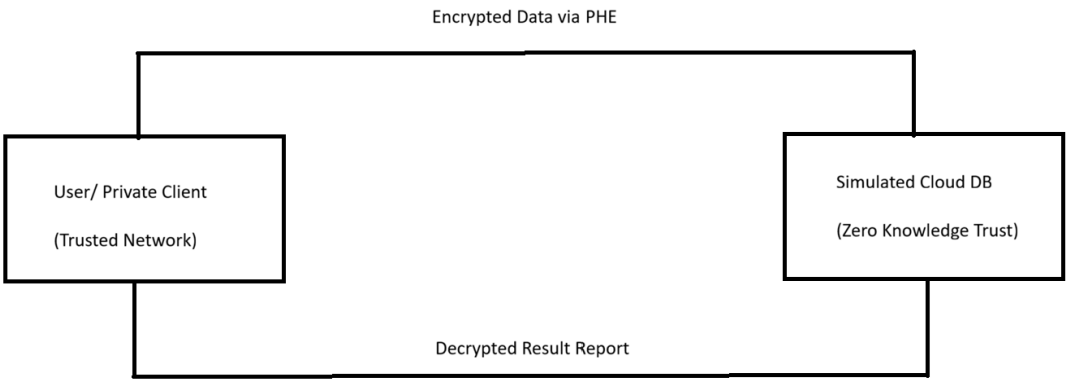
Project motivation & scope?

The project uses Partially Homomorphic Encryption (phe) for the paillier cryptosystem. Fundamentally using the import enable immediate automation creation. Giving the opportunity to bypass having to create a low-level foundation of the mathematical and algorithm modules and engine from scratch.

The idea behind the script simulates the use of fintech possibilities. Running through a local notepad database (salaries.txt) validating and security constraints to then execute blind sum count and data results. I chose this project to bridge the intersection of cybersecurity and finance together, security communication and financial analysis. Exploring how cryptographic methods can solve real world data privacy issues whether it’s finance or even healthcare.

Homomorphic Encryption

Design and Evaluation

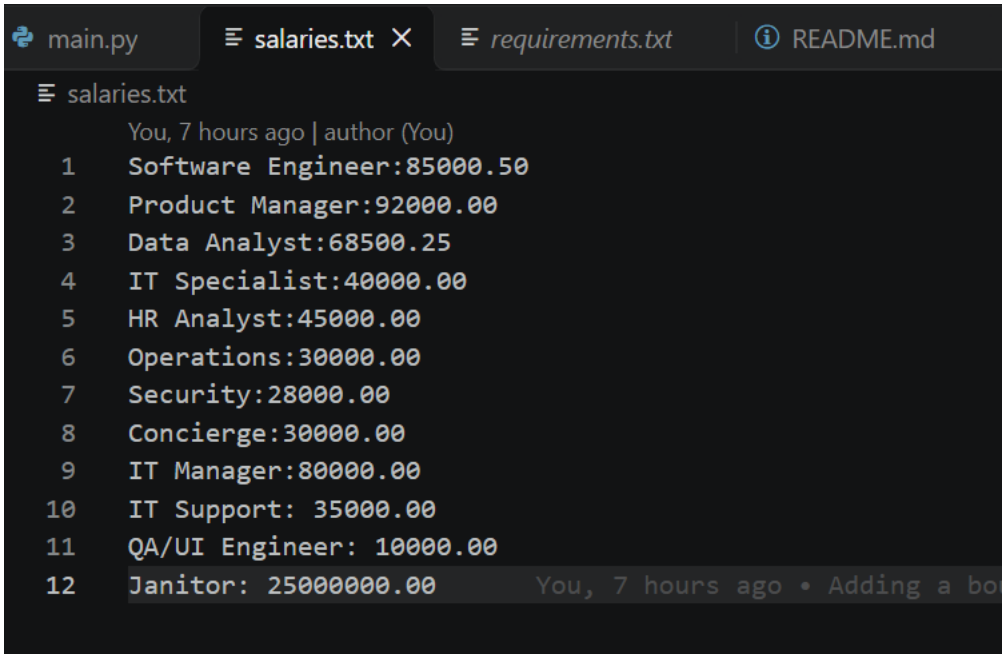


Whilst RAM storage could be a problem due to the key size the main problem and reason the encrypted data is sent to the cloud to run the script is due to the immense CPU power the homomorphic encryption needs to complete a vast mathematical complexity. In order for a single addition loop the CPU has to perform a substantial multiplication on hundreds of strings due to the 2048-bit integer.

When creating my code, I ensured to keep it as close to this diagram design that I created as possible to help ensure for a realistic implementation on how it would work within a company/business.

Testing and Results

Salaries.txt is where I put through information two incorrect via drastically changing the numbers to be too little or too high (out of bounds) and the rest as passable data.



Homomorphic Encryption

The terminal logs are as followed.

```
========== Final Summary ==========
Total Employees: 10
Total Salary: 533500.75
Average Salary: 53350.075
List of Jobs: ['Software Engineer', 'Product Manager', 'Data Analyst', 'IT Specialist', 'HR Analyst', 'Ope
rations', 'Security', 'Concierge', 'IT Manager', 'IT Support']
====================================
```

Key generation test and printed.

```
#This is to print the encrypted salary for each role
cyphertext = encrypted_salaries[role]
print(f"Encrypted salary for {role}: {cyphertext.ciphertext()}")
```

```
Encrypted salary for IT Support: 977353246790494857881720375360909944791392381551576049103378241
314691597873912085981004268568708006287088218660246809413997997065090711610774407438907684369033
070814491408597221784652340808712140954391466601396094515558208928947517176463956167738128215312
111858661230027869144756033793719336640775079937375633709715801297444846724636631354227653004103
107114101143097719289894996783701545780085365178665045122178995546068014397885973854628035947511
096280432264928679276825041331257791569550322830418048675996516948223699243096514395144177867274
842540450680107747104424994551731211398129537470978833625160119670849746063462858731576764221872
939521552126387979322816677367237974379100256272562261052643033568554227796111973585293975146227
060121637672132628387439110000245488955162643636781792263855104969504426855868124904163919423609
```

Although there is clear passable data with zero error on the terminal. With a well seat result from the key generation and the results from the calculations completed for the overall final summary. The data that was out of bounds was skipped showing a total of 10 employees instead of 12 is the only main weakness I came across.